

TuxShow v1.3.0: The Official Technical Manual

Introduction & Environment Setup

TuxShow is a non-linear, hardware-accelerated media server and show control engine designed for live theatrical performance. It is a **Real-Time Compositor**. When you fire a cue, you are not "playing a video file"; you are instructing the GPU to render a WebGL canvas in real-time. This allows you to stack media, manipulate geometry on the fly, and alter show logic based on network triggers without pre-rendering files.

Notice for Windows Users (WSL Configuration)

TuxShow is natively designed for Linux environments. To run TuxShow optimally on a Windows machine, it must be executed through the **Windows Subsystem for Linux (WSL)** with GPU hardware acceleration passed through to the subsystem.

IT Administrator WSL Setup Guide:

1. Open PowerShell as Administrator and run: `wsl --install`
2. Ensure you are running Windows 11 (or Windows 10 version 21H2+) which includes **WSLg** natively for GUI support.
3. Install the specific vGPU driver for your hardware directly on the Windows host (NVIDIA, AMD, or Intel WSL drivers). WSL will automatically mount `/dev/dxg` and pass the hardware acceleration to TuxShow.
4. Run `wsl --update` to ensure the latest graphics kernels are active.

Chapter 1: The Interface Layout & Playback Controls

The TuxShow GUI is divided into distinct zones designed for dark-booth operation.

1.1 The Header & Master Controls

The Header displays your workspace name, GPU status, and global toggles. At the bottom of the interface sits your **Master Control Block**:

- **The Master GO Button (<Enter>):** Executes the currently selected/armed cue.
- **The Pause Button (<Spacebar>):** Freezes all active media and pauses internal engine timers. *Note: When resumed, TuxShow dynamically shifts internal clocks forward, preventing timers from "jumping."*
- **The Panic Button (!):** Instantly executes a master hard-stop. It brutally kills all active media, mutes all audio, and clears the canvas to true black instantly (ignoring any programmed fade-out times).
- **The Record Button (REC):** Captures the live WebGL master canvas output directly to a highly optimized .webm video file on your hard drive. This records the *actual visual output*

(not timeline metadata), allowing you to archive a digital copy of your performance as it happens.

1.2 View Toggles & The Cue List Toolbar

Your master script reads top to bottom. Above the cue list, you have access to View Toggles and the **Cue List Toolbar**, which contains essential management utilities:

- **List / Timeline Toggles (Top Left):** Switch between a standard vertical cue stack and a horizontal temporal graph to spot timing overlap errors.
- **Search / Jump Bar (Magnifying Glass):** Type a cue number and press Enter to instantly scroll the timeline and highlight that specific cue.
- **Unlock All (Open Lock Icon):** A master reset. Instantly clears all locks and re-enables any disabled cues across the entire workspace. Essential for rehearsal resets.
- **Renumber (# Icon):** Automatically recalculates all cue numbers based on their physical top-to-bottom order and folder hierarchy.
- **Add Folder (Folder+ Icon):** Creates a new Group cue to organize your scenes or acts.
- **Add Cue (+ Icon):** Inserts a blank cue immediately below your current selection.
- **Delete (Trash Icon):** Removes the selected cues or folders from the workspace.

1.3 Stage Preview & 4-Tier Inspector

- **Stage Preview:** A WYSIWYG live view of the master canvas. A subtle 3% drafting grid is overlaid on the background to help you easily differentiate the edge of the true-black projection canvas from the UI background.
- **The 4-Tier Inspector (Right Panel):** This context-sensitive menu changes based on the selected cue, split into *Basics/Routing*, *Source*, *Trigger/Timing*, and *Visuals/Effects*.

Chapter 2: System I/O & Hardware Routing

Click the **Settings (Gear Icon)** to configure your booth hardware.

2.1 Audio Driver Selection

TuxShow utilizes Chromium's underlying audio routing layer (WASAPI on Windows, PulseAudio/PipeWire on Linux). To bypass default OS notification sounds, select your physical USB Audio Interface or soundboard under the **Master Audio Device** dropdown. This utilizes the `setSinkId` API for direct hardware routing.

2.2 Global Output Routing & Dedicated Receivers

TuxShow automatically detects connected hardware displays.

- **Global Output:** Select which physical displays (e.g., Display 2 - Projector) the engine should use as an active canvas by default.
- **Dedicated Receiver Mode:** For highly reliable multi-monitor setups, launch a "slave" machine (like a backstage Mac mini). Open a standard web browser, navigate to the WebRTC URL provided in the Master Settings, and it will run a borderless,

hardware-accelerated window that perfectly mirrors the master stream. No native TuxShow installation is required on the slave machine.

2.3 Individual Cue Routing (Multi-Projector Targeting)

If you have multiple projectors connected (e.g., Display 2 mapping Stage Left, Display 3 mapping Stage Right), you can target them individually. In the Inspector's **Basics & Routing** panel, change the **Output Routing** dropdown from All Displays to Display 2. The cue will *only* render on that specific hardware output.

Chapter 3: Media & Trigger Logic

3.1 Trigger Behaviors (The Most Important Lesson)

How a cue behaves when you press "GO" is determined by its Trigger Behavior:

- **Overlap:** The cue plays *on top* of existing active cues. Use this to layer a sound effect over ambient music.
- **Hard-Stop:** Violently clears the entire stage. It forces all other playing cues to instantly stop. Used to cleanly start a brand-new scene.
- **Fade-Target:** Fades a specific playing cue out over a set duration.

3.2 Timing & Auto-Follow Sequences

A 2-second Fade In smoothly ramps audio volume and video opacity from 0 to 100%. If **Follow Action** is set to "Auto-Follow," the engine waits for the cue's Duration to elapse, then automatically triggers the next *enabled* cue.

Chapter 4: Visuals, Geometry & Performance Limits

All transformations use percentage-based math (0 to 100) to ensure your show adapts flawlessly to any projector resolution.

4.1 The Perspective Warp & Custom Masking

- **Perspective Warp:** Drag the four corner pins in the Stage Preview to match the physical corners of slanted set pieces, forcing the image to appear flat to the audience.
- **Masking:** Draw polygons to prevent light spill on proscenium arches. Everything outside the polygon becomes transparent.

4.2 GPU Effect Stacking & Processing Costs

Applying live effects consumes GPU overhead. Understand the processing hierarchy to prevent dropped frames:

- **Low Impact:** Opacity, Scale/Position, Color Correction (HSB Filters).
- **High Impact:** Live Chroma Key, Perspective Warp, Custom Masking.
- **Severe Impact:** Dynamic Drop Shadows / Outlines on high-resolution video. *Warning:*

Stacking Chroma Key + Drop Shadow on multiple simultaneous video cues will bottleneck mid-tier GPUs.

4.3 Animation & SVG Motion Paths

The Animate cue tweens properties of active cues. Enable **Follow Custom SVG Motion Path** to trace complex routes. You can paste a raw SVG coordinate string (e.g., M 10 80 Q 50 10 T 110 80) directly into the Inspector, or draw it on the Canvas.

Chapter 5: Advanced Show Control Utilities

5.1 Nested Groups & Hierarchies

Organizing your show into nested folders is highly recommended.

- **Best Practice:** Create an "Act 1" Group (Fire-First). Inside it, create Scene Groups (e.g., "Scene 1: The Cellar") set to **Fire-All**. This allows you to collapse massive shows into a clean, 10-line view for the operator.

5.2 The State-Changer Cue & Counter Resets

Modifies the metadata of an active cue. If you **Lock** a playing audio track, subsequent Hard-Stop triggers will ignore it.

- **Resetting Counters:** If a director halts rehearsal and asks to run a scene again, sequence counters and locked tracks must be reset. Fire a State-Changer cue set to **Default All States** to instantly unlock tracks and zero out loop counters.

5.3 Conditional Branching (If This, Then That)

Conditional cues evaluate runtime data. If set to **Continuous** mode, the cue acts as a silent background listener that fires its target cue the exact millisecond the condition (like an OSC trigger) becomes true.

5.4 Text & Timer Formatting Syntax

- **Timer Cues:** Support specific formatting strings: MM:SS (05:30), MM:SS.ms (05:30.4), HH:MM:SS (01:05:30), or SS.ms (330.4).
- **Text Cues:** Do not support markdown; they render exact plain-text strings using native Canvas API typography.

Chapter 6: Network Remotes & Security

With Virtual WebRTC enabled in System Settings, TuxShow acts as a local server hosting Progressive Web Apps (PWAs).

6.1 Network Security (Mandatory Practice)

TuxShow endpoints do not utilize PIN codes. Therefore, it is **strictly required** that TuxShow

operates on an isolated, hidden, password-protected "Production Wi-Fi" router. Never host the PWAs on a public school or audience network to prevent unauthorized individuals from triggering cues.

6.2 The Hosted PWAs

- **Live Script (/cuelist):** A dark-themed, read-only sync of the Cue List. As cues fire in the booth, the Stage Manager's tablet instantly highlights active cues via Server-Sent Events (SSE).
- **Stage Manager Deck (/deck):** A remote control surface mimicking the booth's GO/Stop buttons.
- **Game Show Buzzer (/buzzer):** A network trigger for interactive shows or trivia elements.
- **WebRTC Viewer (/viewer.html):** A zero-latency view of the live output, perfect for backstage monitor feeds.
- **Mobile Camera (/camera):** Turns a smartphone into a wireless live camera. *Disconnect Recovery:* If the mobile device drops Wi-Fi mid-show, the WebRTC feed freezes on the last received frame (preventing an abrupt black screen) and resumes instantly upon reconnection.

6.3 Console Integration Protocols

| OSC Path | Arguments | Description |

| /tuxshow/go | None | Executes the master "GO" button. |

| /tuxshow/stop | None | Executes a "Stop All" (Hard Stop). |

| /tuxshow/select/cue | [String] (Cue Number) | Moves selection to a specific cue (e.g., 1.5). |

| /tuxshow/master/volume | [Float] (0.0 to 1.0) | Adjusts global master volume. |

| /tuxshow/cue/[num]/start | None | Forces a specific cue number to fire. |

- **Custom OSC Listeners:** Create a Conditional cue set to **Continuous**, instruct it to listen for a custom path (e.g., /touchosc/fader1 sent from the TouchOSC app), and trigger a specific sequence when it arrives.
- **MSC:** TuxShow listens for standard legacy MSC hex packets (Commands: GO, STOP, RESUME, LOAD).

Chapter 7: Optimization & Show Prep

7.1 Mitigating DOM Bloat (RAM Management)

While WebGL is efficient, having 1,000 cues loaded simultaneously consumes vast amounts of system RAM.

- **Best Practice for Heavy Shows:** For massive 2-Act musicals containing hundreds of video files, program your show as two entirely separate workspace files (Show_Act1.tsw and Show_Act2.tsw). During intermission, click "Clear Workspace" and load Act 2 to flush

the system memory.

7.2 Packing the Workspace

TuxShow .tsw files use absolute file paths. **Never manually move your showfile.** Always click the **Pack** button. This algorithm gathers every video, image, and audio file, re-links the paths to relative URLs, and creates a portable folder.

Appendix: Building a Show Walkthrough (*Really Rosie*)

Bridging theoretical knowledge into practical execution.

1. Storyboarding & Organization: We started offline, defining exact moments when the mid-20th-century Sendak-style illustrations needed to appear. We organized the main list into "Act 1", "Act 2", nested with "Scene" folders to keep the operator's screen clean.

2. The Overture (Linear Automation): We placed an Auto-Start Conditional cue listening for the OSC string /lighting/house/out. When the lighting board faded the house lights to 0%, the Overture audio triggered automatically. The Title sequence was placed in a **Fire-All** group, stacking the .mp3 over a .png graphic using the Overlap behavior.

3. The Z-Axis "Push In" (Avenue P Stoop): To make a flat 2D graphic feel cinematic, we targeted the Brownstone backdrop with an Animate cue. By tweening both scaleX and scaleY from 100% to 150% over a 15-second duration, it created a slow, creepy "camera push in" effect.

4. The Sliding Canvas (Chicken Soup with Rice): To simulate a flip-book or carousel, we built a nested group containing the "January", "February", and "March" graphics. As "January" finished, an Animate cue rapidly pushed its posX to -50% (offscreen left), while simultaneously triggering "February" to slide in from 150% (offscreen right).

5. The Cellar Storm (Non-Linear Looping):

We needed infinite lightning that felt organic, not robotic.

- Cue 18.2 (Pause): 1.0s duration (Auto-Follow ON) to pace the lightning.
- Cue 18.3 (GoTo Random): Pointed at Cues 18.4 - 18.8.
- Cues 18.4 to 18.8 (Image): Five different Cellar.png graphics, heavily overexposed using the HSB Color Correction filter to create a bright flash.
- **The Return Path:** Placed immediately after every flash was a GoTo pointing back to Cue 18.2. This trapped the engine in an infinite, random loop until the operator fired Scene 19 (set to Hard-Stop), cleanly breaking the loop.

6. Rehearsal Safety Macros: At the top of Scene 19, we added a State-Changer cue set to **Default All States**. When the director constantly yelled "Go back and run the Cellar storm again!" the operator simply hit this macro to instantly unlock any audio tracks, clear any running logic, and reset the loop counters perfectly.

Acknowledgments & Licensing

Acknowledgments

With deepest gratitude for years of support, knowledge, and friendship:

Shawna, Madysun, JD, and little Charley (welcome to the world!)

And to my mentors for their guidance:

Tony, Jeff, Mark, Leon, and Squeek!

Licensing (MIT)

Copyright (c) 2026 Christopher Earl Baker

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.